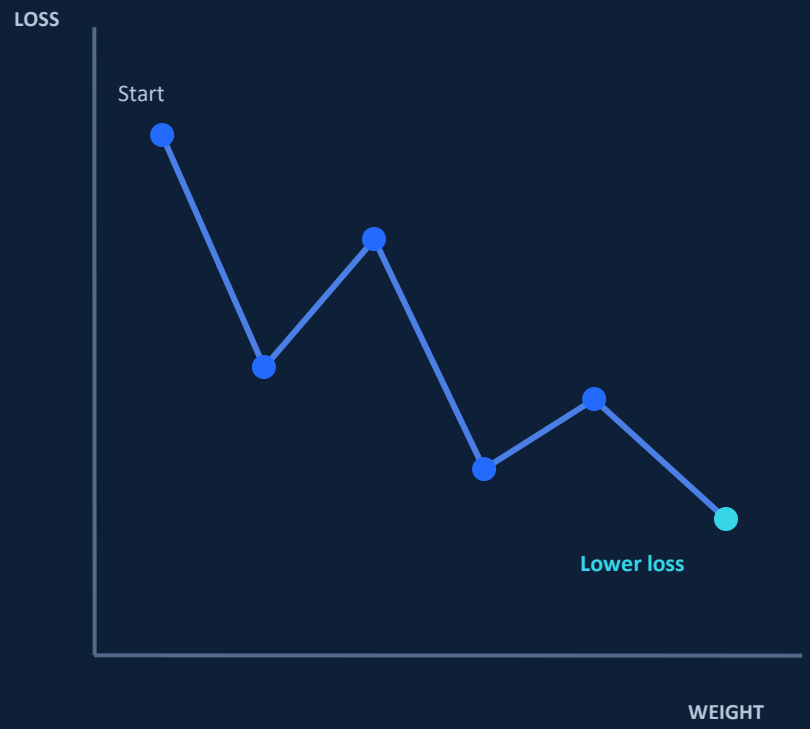
STOCHASTIC GRADIENT DESCENT CLASSIFIER

Understanding SGD optimization and classification with Scikit-learn.

> SGD is an optimization method. SGDClassifier uses sample-level gradients to learn a linear classifier.

WEIGHT UPDATES TOWARD LOWER LOSS



$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t)$$

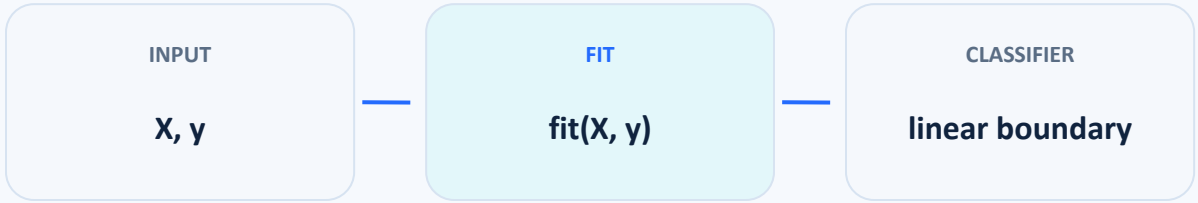
What is SGDClassifier?

A linear classifier trained with stochastic gradient descent.

- **Predicts class labels**
  For example: Low, Medium or High electricity consumption.
- **Supports binary and multiclass tasks**
  String labels such as Low, Medium and High can be used directly.
- **Learns a linear decision boundary**
  The model updates coefficients and intercept during training.

X = features **y = target class**

FIT AND PREDICT



predict(X)
returns class labels

predict_proba(X)
available with log_loss

SGD optimization algorithm

SGDClassifier Scikit-learn classification estimator using SGD

Classification losses: hinge and log loss

Loss measures mistakes during training. Evaluation metrics measure model performance afterward.

Loss	Model behaviour	Probability support	Typical use
hinge (default)	Enforces a margin around the decision boundary	No predict_proba()	Linear SVM-style classification
log_loss	Optimizes class probabilities; penalizes confident errors	predict_proba() available	Logistic-regression-style classification

Hinge loss Focuses on correct classification with a margin.

Log loss Uses predicted probabilities and penalizes confident mistakes.

Choose the loss to match the prediction output your application needs.

Cross-entropy (log loss)

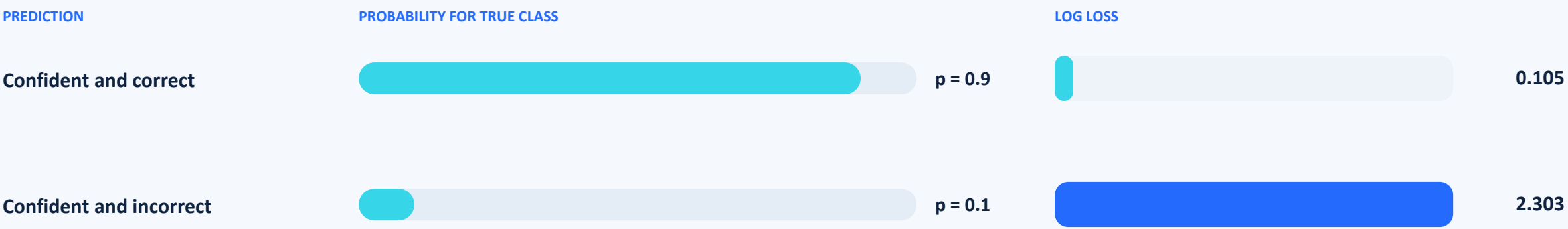
For binary classification, log loss compares the true label with the predicted probability.

BINARY CROSS-ENTROPY

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

When the true label is y = 1

The loss becomes −log(p). Higher probability on the true class means lower loss.



Correct, confident predictions receive low loss. Confident errors receive high loss. SGDClassifier calls this option loss="log_loss".

Parameters, regularization and training

Scale numerical features, choose a loss, then tune the optimization settings.

KEY PARAMETERS

- **loss** — hinge or log_loss
- **penalty / alpha** — regularization type / strength
- **l1_ratio** — L1 share for elastic net
- **learning_rate / eta0** — schedule / initial step size
- **max_iter / tol** — epoch limit / stopping
- **random_state** — repeatable shuffling

REGULARIZATION

L1 can zero coefficients. L2 shrinks them. Elastic Net combines both. alpha sets strength; l1_ratio sets the L1/L2 mixture for elasticnet.

Scaling matters

Fit the scaler on training data, then transform both splits with the same scaler.

predict_proba() works with log_loss. It is unavailable with default hinge loss.

PYTHON / SCIKIT-LEARN

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = SGDClassifier(
    loss='log_loss', penalty='l2', alpha=0.0001,
    learning_rate='constant', eta0=0.01,
    max_iter=1000, random_state=42
)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)
```

TRAIN, TUNE AND EVALUATE SGDCLASSIFIER

END-TO-END WORKFLOW

1. Prepare data
2. Split train / test
3. Encode inputs + scale
4. Train classifier
5. Tune with GridSearchCV
6. Evaluate

EVALUATION METRICS

- Confusion matrix
- Accuracy
- Precision
- Recall
- F1-score
- Classification report

GRIDSEARCHCV

best_params_	selected settings
best_estimator_	refitted best model
best_score_	best CV score

Understand the loss → Optimize the model → Predict the classes → Evaluate performance.